

shogiAI 數據科學報告

資料前處理、訓練邏輯、模型成果與 AI 設計

shogiAI 專案

2026 年 5 月

摘要

本報告說明 shogiAI 專案的數據科學流程。專案以 CSA 將棋棋譜作為資料來源，經過解析、合法性驗證、局面編碼、走法標籤化、勝負標籤建立與資料集切分後，訓練一個 policy/value 卷積神經網路。模型輸入為 $43 \times 9 \times 9$ 的棋盤張量，輸出包含 13,689 維 policy logits 與 $[-1, 1]$ 的 value 預測。訓練後模型再與 alpha-beta 搜尋、啟發式評估、開局庫、走法排序、transposition table 等模組結合，形成可實際下棋與可評估模型強度的 AI 系統。

目录

1	資料科學目標	2
2	資料來源與規模	2
3	資料前處理流程	3
3.1	解析棋譜	3
3.2	局面編碼	3
3.3	行棋方視角旋轉	3
3.4	走法標籤化	4
3.5	Value 標籤	4
4	資料處理邏輯與原理	4
4.1	為什麼用 supervised learning	4
4.2	為什麼保存 metadata	5
4.3	為什麼用 game-level split	5
5	模型架構	5
5.1	Feature Extractor	5

目錄	2
5.2 Policy Head	5
5.3 Value Head	5
6 訓練方法	6
6.1 Loss Function	6
6.2 Optimizer 與 Scheduler	7
7 訓練成果	7
7.1 成果解讀	7
8 AI 設計	8
8.1 傳統搜尋核心	8
8.2 手工評估函數	8
8.3 Policy 網路如何介入搜尋	9
8.4 Value 網路如何介入搜尋	9
8.5 開局庫	9
9 模型評估與比賽	9
10 限制與改進方向	10
10.1 資料限制	10
10.2 模型限制	10
10.3 Value label 限制	10
10.4 評估限制	10
11 教授可能提問與回答	10
11.1 為什麼不用 one-hot 表示整個棋盤就好	10
11.2 為什麼 policy 輸出 13,689 類，而不是只輸出合法手	10
11.3 Top-1 只有 33%，是否太低	10
11.4 為什麼還需要 alpha-beta，不直接用模型下	11
11.5 Value head 為什麼效果較差	11
12 結論	11
A 常用指令	11
A.1 匯出訓練資料	11
A.2 訓練模型	11
A.3 模型對弈測試	12

1 資料科學目標

本專案的資料科學目標是讓 AI 從人類實戰棋譜中學習兩件事：

1. **Policy**：在某個局面下，實戰高手下一步通常會怎麼走。
2. **Value**：在某個局面下，輪到行棋的一方最後贏棋的可能性。

模型可表示為：

$$f_{\theta}(s) = (p, v)$$

其中 s 是局面， p 是所有可能走法的分數， v 是局面勝負價值。Policy 用於排序候選手，Value 用於輔助局面評估。這兩者再與傳統 alpha-beta 搜尋結合，形成混合式 AI。

2 資料來源與規模

資料來源為 data/ 目錄與 MySQL 中的 CSA 棋譜。CSA 是將棋常用棋譜格式，包含棋手、賽事、日期、初始局面、走法、耗時、註解與結果。

目前匯出的訓練資料集為 out/policy_dataset.npz，實際統計如下：

項目	數值
對局數	7,305 局
訓練樣本數	832,773 筆
狀態張量形狀	(832773, 43, 9, 9)
Policy 走法類別數	13,689 類
Value 有標籤樣本	832,773 筆
Value 無標籤樣本	0 筆
是否以行棋方視角旋轉	True

結果分布如下：

結果	樣本數
TORYO	831,333
DRAW	104
SENNICHITE	1,336

3 資料前處理流程

資料前處理由 `src/csa_preprocess.py` 與 `src/export_policy_dataset.py` 負責。整體流程如下：

CSA / MySQL $\rightarrow$ `cshogi.Board` $\rightarrow$ state tensor
 $\rightarrow$ move label $\rightarrow$ value label $\rightarrow$ NPZ dataset

3.1 解析棋譜

CSA 棋譜先由 `cshogi.CSA.Parser` 解析。若來源是 MySQL，系統會從 `positions` 與 `moves` 取出每一筆「局面加下一手」資料。解析後以 `cshogi.Board` 重建局面：

```
board = cshogi.Board(sfen)
move = board.move_from_usi(usi_move)
board.is_legal(move)
```

若某一步不合法，該筆資料不會進入訓練集。這一步是資料品質控制的核心。

3.2 局面編碼

神經網路不能直接吃 SFEN 字串，因此系統將棋盤轉成固定大小張量：

$$state \in \mathbb{R}^{43 \times 9 \times 9}$$

43 個 planes 的設計如下：

Plane 範圍	意義
0–13	自方 14 種棋子在棋盤上的位置。
14–27	對方 14 種棋子在棋盤上的位置。
28–34	自方持駒數量，7 種持駒各一個 plane。
35–41	對方持駒數量，7 種持駒各一個 plane。
42	行棋方 plane。使用行棋方視角時通常為常數。

這種設計讓 CNN 可以把棋子位置、持駒資訊與行棋方資訊視為影像通道處理。

3.3 行棋方視角旋轉

專案預設 `orient_to_turn=True`。當輪到後手行棋時，棋盤會旋轉 180 度，使模型永遠從「自己要下」的角度看局面。

這個設計有三個好處：

1. 減少先手與後手對稱造成的學習負擔。

2. 讓同樣戰型在不同顏色下更容易被模型視為相似資料。
3. 讓 value label 可以統一解讀為「目前行棋方是否有利」。

3.4 走法標籤化

Policy head 需要把每個合法走法對應到一個整數 label。將棋走法分成一般移動與打入持駒。

一般移動考慮來源格、目標格與是否升變：

$$81 \times 81 \times 2 = 13,122$$

打入持駒考慮 7 種可打入棋子與 81 個目標格：

$$7 \times 81 = 567$$

因此總類別數為：

$$13,122 + 567 = 13,689$$

這就是模型 policy 輸出的維度。訓練時使用 CrossEntropyLoss，目標是讓實戰下一手的 label 分數最高。

3.5 Value 標籤

Value label 表示目前行棋方最後的勝負結果：

$$z = \begin{cases} 1, & \text{目前行棋方最後獲勝} \\ 0, & \text{和棋或千日手} \\ -1, & \text{目前行棋方最後落敗} \end{cases}$$

若結果是 TORYO，系統根據最終局面的行棋方推定投降方與勝方；若是 DRAW 或 SENNICHITE，則標記為 0。

4 資料處理邏輯與原理

4.1 為什麼用 supervised learning

專案目前的資料是實戰棋譜，最直接的學習方式是模仿學習：把每個局面的人類下一手當作 policy 標籤，把最終勝負當作 value 標籤。這種方法不需要先讓 AI 自我對弈，即可快速得到可用的走法排序模型。

4.2 為什麼保存 metadata

NPZ 中的 meta 會保存 game_id、ply、sfen、move_usi、result 等資訊。這讓每筆樣本都能追溯，方便除錯與回答「這筆訓練資料從哪盤棋來」。

4.3 為什麼用 game-level split

訓練集、驗證集、測試集不是隨機切單手，而是按 game_id 分割。原因是同一盤棋相鄰局面非常相似，如果同一盤棋的前半出現在訓練集、後半出現在測試集，測試結果會過度樂觀。因此專案使用 game-level split 降低資料洩漏。

5 模型架構

模型定義於 src/policy_model.py，名稱為 SmallPolicyValueNet。模型分成共同特徵抽取器、policy head 與 value head。

5.1 Feature Extractor

```
Conv2d(43, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 64, kernel_size=3, padding=1)
ReLU
Conv2d(64, 32, kernel_size=3, padding=1)
ReLU
```

卷積層保留 9×9 空間結構，適合學習棋子局部關係，例如攻防、保護、升變區與棋子協同。

5.2 Policy Head

```
Flatten
Linear(32 * 9 * 9, 512)
ReLU
Linear(512, 13689)
```

Policy head 輸出 13,689 維 logits。推論時只取合法手對應的 label 分數，再排序合法手，避免 AI 選到不合法走法。

5.3 Value Head

```

Flatten
Linear(32 * 9 * 9, 128)
ReLU
Linear(128, 1)
Tanh

```

最後使用 Tanh，使輸出落在 $[-1, 1]$ ，與 value label 的範圍一致。

6 訓練方法

訓練腳本為 `src/train_policy.py`。資料切分與訓練設定如下：

項目	數值
Train samples	667,345
Validation samples	81,882
Test samples	83,546
Optimizer	AdamW
Learning rate	0.001
Minimum learning rate	0.00001
Weight decay	0.0001
Epochs	12
Value loss weight	0.25

6.1 Loss Function

總損失為：

$$\mathcal{L} = \mathcal{L}_{policy} + \lambda \mathcal{L}_{value}$$

其中 policy loss 使用 cross entropy：

$$\mathcal{L}_{policy} = -\log \frac{\exp(\hat{p}_a)}{\sum_j \exp(\hat{p}_j)}$$

Value loss 使用 MSE，並乘上 value mask：

$$\mathcal{L}_{value} = m(\hat{v} - z)^2$$

本次資料集所有樣本都有 value label，因此 value mask 全部為 1。保留 mask 設計是為了未來支援沒有明確勝負結果的資料。

6.2 Optimizer 與 Scheduler

專案使用 AdamW 與 CosineAnnealingLR。AdamW 將權重衰減與 Adam 更新分離，比傳統 Adam 加 L2 regularization 更穩定；Cosine annealing 則讓 learning rate 從較大值逐漸降低，後期訓練更細緻。

7 訓練成果

最新 checkpoint out/policy_model.pt 的測試集成果如下：

指標	數值
Test policy loss	3.0271
Top-1 accuracy	0.3337
Top-3 accuracy	0.5381
Top-5 accuracy	0.6244
Value loss	1.0201
Value MAE	0.9425

最後一個 epoch 的 validation 指標如下：

指標	數值
Train loss	1.8179
Validation policy loss	3.3369
Validation Top-1 accuracy	0.3500
Validation Top-3 accuracy	0.5566
Validation Top-5 accuracy	0.6392
Validation value loss	1.1268
Validation value MAE	0.9134

7.1 成果解讀

Top-1 accuracy 約 33.37%，代表模型直接猜中實戰下一手的比例。將棋分支因子高，而且同一局面可能有多個合理手，所以 Top-3 與 Top-5 更能反映 policy 是否把合理候選手排在前面。Top-5 約 62.44%，表示模型已能提供有價值的候選手排序，適合用來輔助 alpha-beta 搜尋。

Value MAE 約 0.94，代表 value head 仍有改進空間。原因可能包含：只用最終勝負標籤會使中盤局面的 value 訊號較粗糙；投降資料會讓最後勝負明確，但中間局面未必能直接判斷；模型架構較小，對複雜局面估值能力有限。

8 AI 設計

shogiAI 不是只用神經網路直接選最大 policy 的走法，而是將模型與搜尋結合。核心模組位於 `src/ai_search.py`、`src/policy_model.py` 與 `src/csa_browser_api.py`。

8.1 傳統搜尋核心

AI 使用 negamax 形式的 alpha-beta search：

$$score(s) = \max_a -score(T(s, a))$$

搜尋模組包含：

- **Alpha-beta pruning**：剪掉不可能影響結果的分支。
- **Quiescence search**：在葉節點延伸吃子、升變等戰術手，降低 horizon effect。
- **Transposition table**：用 Zobrist hash 快取已搜尋局面。
- **Killer move**：記錄同層造成 beta cutoff 的安靜手。
- **History heuristic**：累積過去有效走法的排序分數。
- **Late move reduction**：對排序靠後且安靜的走法降低搜尋深度。

8.2 手工評估函數

`evaluate_position` 不只計算子力，還加入位置與戰術特徵：

- 子力價值與持駒價值。
- 棋子位置分數，例如推進與中央控制。
- 升變區獎勵。
- 機動力。
- 王安全與周圍保護。
- 懸空棋子懲罰。
- 持駒對敵王周圍的壓力。
- 節奏 bonus。

這些特徵讓 AI 即使沒有神經網路，也能用傳統方法下出合理棋。

8.3 Policy 網路如何介入搜尋

Policy 網路主要用於 move ordering。流程如下：

legal moves $\rightarrow$ policy logits $\rightarrow$ sorted candidate moves $\rightarrow$ alpha-beta search

Alpha-beta 的效率高度依賴走法排序。如果好手排在前面，就更容易提早產生 cut-off，搜尋同樣深度時可少看很多節點。因此目前模型的 Top-5 表現對 AI 實戰很有價值。

8.4 Value 網路如何介入搜尋

Value head 可用於局面加權：

$$\text{score} = \text{handcrafted evaluation} + w \cdot V_{\theta}(s)$$

--value-weight 控制 value head 影響力。若設為 0，代表只用傳統評估；若設為 100 或 200，代表讓神經網路估值參與根節點或局面評估。實務上 value head 目前仍較粗糙，所以需要謹慎調整權重。

8.5 開局庫

AI 可從 MySQL 建立 opening book。系統統計前若干手每個 SFEN 對應的常見下一手，如果某手出現次數達到門檻，就可作為開局走法。這能讓 AI 在開局階段更接近實戰棋譜，降低早期搜尋成本。

9 模型評估與比賽

src/compare_ai_models.py 可讓兩個模型對弈。比賽時會輪流交換先後手，避免先手優勢造成偏差。輸出包含：

- 新模型勝場。
- 舊模型勝場。
- 和棋數。
- 新模型得分率。
- 平均手數。
- 每盤棋的 USI 手順。

目前 smoke test 中 1 盤短對局結果為和棋，得分率 0.5，主要用於驗證流程能跑通，不代表完整棋力評估。若要更可靠，需要增加對局數、搜尋深度與時間限制。

10 限制與改進方向

10.1 資料限制

目前資料來自實戰棋譜，品質依賴原始 CSA 是否完整。非法棋譜會被過濾，但對局結果與投降時間仍可能影響 value 標籤品質。

10.2 模型限制

目前模型是小型 CNN，沒有 residual block 或 attention。它適合快速訓練與作為 move ordering，但若要追求更高棋力，可改成 residual CNN、Transformer 或加入合法手 mask 的 policy loss。

10.3 Value label 限制

Value label 直接取最終勝負，對中盤局面可能太粗糙。例如某方最後獲勝，不代表中途每一個局面都明顯有利。未來可以加入搜尋評分、自我對弈結果或多步 bootstrap 方式改良 value target。

10.4 評估限制

Top-k accuracy 衡量的是模仿人類下一手，不完全等於棋力。真正棋力仍需靠模型對弈、固定時間測試與更多局數統計。

11 教授可能提問與回答

11.1 為什麼不用 one-hot 表示整個棋盤就好

本專案其實就是把棋盤轉為多通道 one-hot 與數量 plane。不同棋種與持駒分開 channel，可以讓 CNN 更容易學到「哪種棋子在哪裡」以及「持駒會造成什麼威脅」。

11.2 為什麼 policy 輸出 13,689 類，而不是只輸出合法手

神經網路輸出維度需要固定，但每個局面的合法手數量不同。因此模型輸出所有可能走法的固定 label，推論時再只挑合法手。這是棋類 policy network 常見做法。

11.3 Top-1 只有 33%，是否太低

將棋每個局面有很多合法手，而且多個走法都可能合理。模型目標是提供候選手排序，不一定要直接取代搜尋。Top-5 約 62%，代表它能把實戰合理手放進前幾名，對 alpha-beta move ordering 已經有幫助。

11.4 為什麼還需要 alpha-beta，不直接用模型下

只用模型會缺少戰術計算，容易漏看將死、吃子交換與短期陷阱。Alpha-beta 能計算變化，policy/value 則幫助排序與估值。混合設計比單獨使用其中一種更穩定。

11.5 Value head 為什麼效果較差

Value 只用最終勝負監督，而中盤到終局距離很長，訊號雜訊比 policy 更高。此外小型 CNN 容量有限，難以完全評估複雜局面。改進方式包含更多資料、更深模型、自我對弈與搜尋產生的評估標籤。

12 結論

shogiAI 的數據科學流程從原始 CSA 棋譜出發，經由合法性驗證、局面張量化、走法標籤化與勝負標籤化，建立 832,773 筆 policy/value 訓練樣本。模型以小型 CNN 同時預測下一手與局面價值，測試 Top-1 accuracy 約 33.37%，Top-5 accuracy 約 62.44%。在 AI 設計上，模型不是孤立使用，而是與 alpha-beta 搜尋、手工評估函數、開局庫與多種搜尋最佳化結合。這使專案同時具備資料處理、模型訓練、棋力搜尋與前端互動展示的完整流程。

A 常用指令

A.1 匯出訓練資料

```
python src\export_policy_dataset.py ^
--output out\policy_dataset.npz ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213
```

A.2 訓練模型

```
python src\train_policy.py ^
--input out\policy_dataset.npz ^
--output out\policy_model.pt ^
--batch-size 256 ^
--value-loss-weight 0.25
```

A.3 模型對弈測試

```
python src\compare_ai_models.py ^  
  --old-model out\policy_model.prev.pt ^  
  --new-model out\policy_model.pt ^  
  --games 20 ^  
  --depth 3 ^  
  --time-limit-ms 1000
```